

HarborOps Picks the Wrong Corpus

Table of Contents

1. [Learning Objectives](#)
2. [Case Pipeline](#)
3. [Thursday, 6:40 p.m., and the pilot was pointed at the wrong data](#)
4. [What the team learned in six uneasy weeks](#)
5. [Exhibits: source contract evidence and architecture boundaries](#)
6. [Questions before you read the recommendation](#)
7. [Model answers Priya would want to hear](#)
8. [What to carry into your own capstone](#)
9. [Decision Points](#)
10. [Analysis Walkthrough](#)

 Duration: 30 minutes

Learning Objectives

- Diagnose whether a reachable source is actually AI-ready for a governed retrieval product.
- Choose a right-sized platform architecture by working backward from the curated data product and its consumers.
- Classify candidate sources as usable, usable with controls, or out of scope based on operational-contract evidence.
- Defend an ADR-backed decision between warehouse-first and lakehouse-first patterns under time and governance constraints.

- Design control boundaries for a controlled corpus pipeline, including human review, lineage, and quality gates.

Case Pipeline

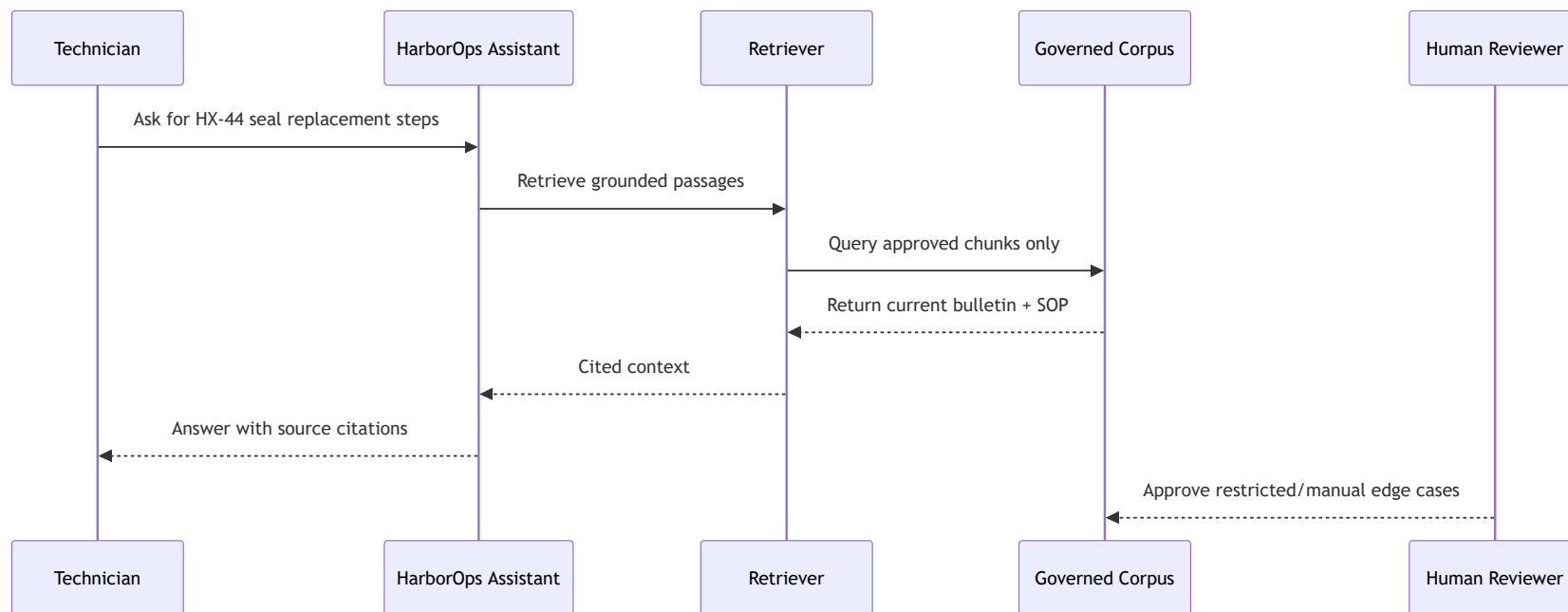


Thursday, 6:40 p.m., and the pilot was pointed at the wrong data

On **Sep 26, 2024**, Priya Natarajan was three slides into a design review when the field-service VP stopped her. "Show me where the assistant gets the actual service bulletin, not the ticket summary." Silence. The current plan used Snowflake tables from maintenance tickets and ERP assets, plus a generic vector add-on bolted on top. It could retrieve failure codes. It could not reliably retrieve the bulletin that changed the repair procedure for a pressure valve two weeks earlier.

The stakes were not academic. HarborOps maintains port equipment across **14 terminals**. The Q4 pilot covered **620 technicians**, with a target to cut average diagnostic time by **18%** and improve first-call fix rate from **71% to 78%**. A wrong answer did not just waste time; it could put a crane or pump back into service with the wrong part or procedure.

Priya had to make a call that night: keep the simpler warehouse-first design and hope structured data was enough, or reset around a governed corpus and a right-sized RAG pipeline. She had two weeks before procurement froze Q4 scope.



What the team learned in six uneasy weeks

- **Aug 12, 2024** — HarborOps approved the use case: an internal technician assistant for repair guidance, with citations required and no direct internet retrieval allowed.
- **Aug 19, 2024** — The data platform team inventoried four candidate sources: maintenance ticket export, ERP asset master, SharePoint manuals repository, and IoT telemetry stream.
- **Aug 28, 2024** — A quick prototype on ticket text achieved only **46% grounded-answer pass rate** on 50 technician questions; most misses required a service manual or bulletin not present in tickets.
- **Sep 3, 2024** — Security flagged that **11%** of maintenance SOP attachments contained personal phone numbers or signatures; privacy controls became mandatory for any document-serving path.
- **Sep 11, 2024** — OCR tests on a 2,000-document sample from SharePoint showed **86% extractable text coverage**, but scan quality failures clustered in older vendor PDFs from 2017-2019.
- **Sep 18, 2024** — Priya drafted two ADRs: one for warehouse-first with vector add-on, one for lakehouse-first governed corpus.

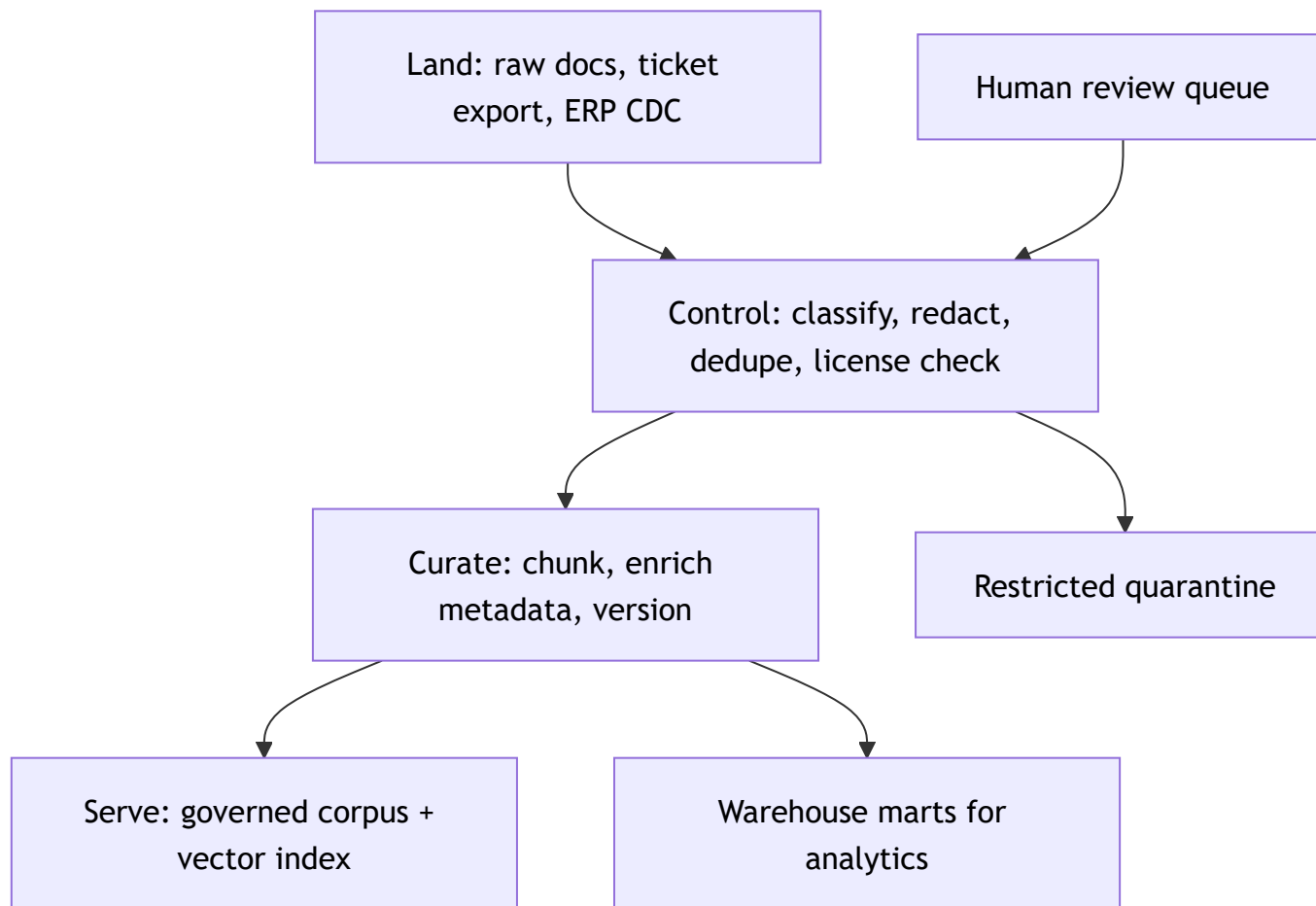
Neither yet resolved who owned document freshness and license review.

- **Sep 26, 2024** — The VP challenged the design review after seeing that the architecture centered on tools already in place rather than on the curated product technicians actually needed.

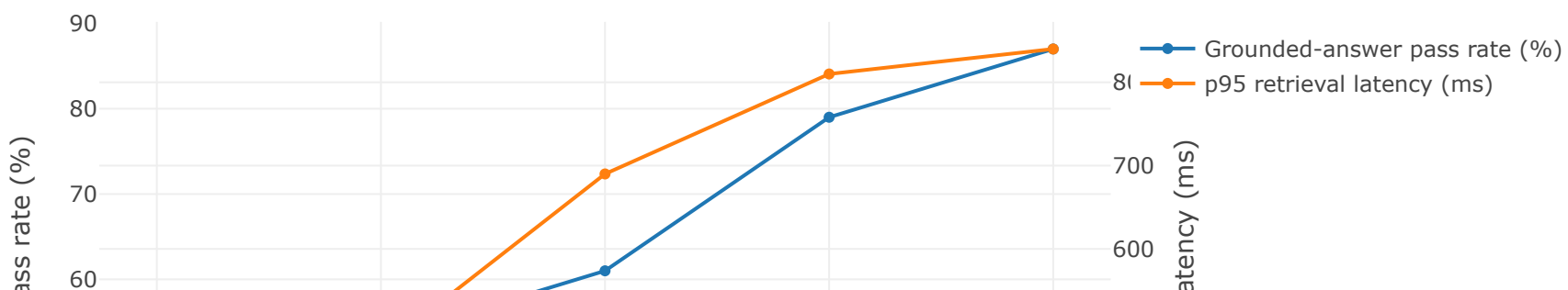
Exhibits: source contract evidence and architecture boundaries

Source	Type	Owner	Update pattern	Reachable?	Business usefulness	AI usefulness	Key failure mode	Decision
Maintenance ticket export	Structured + free text	Service Ops BI team	Nightly batch, 02:00 UTC	Yes	High for trends, parts, failure codes	Medium for retrieval context; weak for authoritative procedures	Technician notes are terse and inconsistent	Usable
ERP asset master	Structured	Enterprise Apps	Near-real-time CDC every 15 min	Yes	High for asset identity and installed base	Low alone; useful as filters and metadata	Asset aliases drift by terminal	Usable

SharePoint manuals repository	Unstructured PDFs/DOCX	3 regional maintenance leads	Irregular; bursty after vendor releases	Yes, messy	High; contains official SOPs, bulletins, manuals	High if governed and chunked	License ambiguity, OCR failures, stale duplicates	Usable with controls
IoT telemetry stream	Time series events	Platform Ops	5-second events	Yes	Medium for anomaly context	Low for current assistant scope	No procedure content; high storage cost	Out of scope for phase 1



Prototype results after adding governed document content





Why it matters: The chart is not celebrating model quality in the abstract. It shows that answer quality rose only after the document repository entered the pipeline, even though latency also rose from **420 ms** to **840 ms**. HarborOps had to decide whether that extra complexity bought the right kind of value.

Questions before you read the recommendation

1. **[Analyze]** Which source is most dangerous to include without controls, and which evidence in the exhibits makes that clear?
2. **[Evaluate]** If the committed product is a technician assistant with citations, which decision option best matches the consumer need and why does at least one other option fail even if it is easier to build?
3. **[Apply]** Based on the timeline and chart, where would you place the human-review boundary in the pipeline: before curation, after embedding, or only at serving time?
4. **[Analyze]** Should HarborOps classify IoT telemetry as unusable, usable with controls, or out of scope for phase 1? Use the module's distinction between business usefulness and AI usefulness.
5. **[Evaluate]** What should the ADR explicitly record so that this design stays implementation-ready without drifting into production-scale overdesign?

Model answers Priya would want to hear

For **Question 1**, the SharePoint manuals repository is the risky one, not because it is bad, but because it is powerful and uncontrolled. The table shows **high AI usefulness** paired with license ambiguity, stale duplicates, and OCR failure risk. That is a

textbook **usable with controls** source. Option 4 ignores those controls; Option 2 turns them into explicit pipeline stages.

For **Question 2**, **Option 2** is the best fit because the product requires authoritative, citable content. The ticket export and ERP tables help, but the chart shows grounded-answer pass rate did not approach viability until document content was added, climbing from **46%** to **87%**. **Option 1** is easier but mismatched to the retrieval product. **Option 3** could be a good separate capstone, but it misses the committed consumer need.

For **Question 3**, put human review **before curation is released to serve**, not only at the end. Review belongs after landing and initial control checks, where restricted or low-quality documents can be quarantined before chunking and indexing. The diagram's boundary between **Control** and **Curate** is the governance fulcrum. Waiting until serving time is too late; unsafe chunks would already be embedded and retrievable.

For **Question 4**, IoT telemetry is best marked **out of scope for phase 1**, not globally unusable. It has some business usefulness for anomaly context, but the assistant's retrieval task is procedure-heavy, and telemetry carries almost no direct answer content. The case numbers also hint at cost pressure. A right-sized scope resists ingesting a high-volume stream just because it is available.

For **Question 5**, the ADR should record the product target, accepted sources and classifications, chosen architecture pattern, control boundaries, rejected alternatives, validation plan, and one explicit non-goal. In this case the non-goal matters: HarborOps is not building a full enterprise knowledge mesh or a training-data release in Q4. The design should be buildable next, with evidence gates like **85% grounded-answer pass rate** and **zero restricted-document leakage**.

What to carry into your own capstone

If your own platform project has one source everyone loves because it is easy to reach, ask the HarborOps question: does it actually contain the semantic material your AI component needs, or is it merely convenient?

- **Start** from the curated product and named consumer, then work backward to sources and storage.
- **Classify** each source as usable, usable with controls, or out of scope using ownership, update pattern, and failure mode evidence.
- **Separate** operational reachability from AI fitness; a clean API can still be the wrong source for retrieval or training.

- **Draw** state changes and control boundaries explicitly: raw, controlled, curated, served, restricted, human-reviewed.
- **Right-size** the architecture so the next build is obvious; avoid speculative infrastructure that the current product does not need.
- **Write** ADRs that include alternatives, consequences, governance implications, and falsifiable validation checks.

Decision Points

Option 1. Build the assistant on the maintenance ticket export and ERP tables only, keeping the platform warehouse-first.

- **Expected impact:** This ships fastest for analytics reuse but will likely produce weak retrieval because the structured sources lack procedure detail and document-level grounding.
- **Key risk:** The team confuses business relevance with AI relevance and locks in a reachable but retrieval-poor corpus.
- **Evidence to confirm:** Pilot retrieval tests hit a grounded-answer pass rate of at least 82% on 50 technician questions using only ticket and ERP fields.
- **Evidence to kill:** Technician answers repeatedly require PDF procedures or scanned bulletins that are absent from the structured sources.

Option 2. Adopt a lakehouse-first controlled corpus with a governed RAG pipeline, using the document repository as usable with controls and tickets as supporting context.

- **Expected impact:** This creates the highest chance of grounded answers and auditable lineage while preserving room for later analytics products.
- **Key risk:** The team takes on OCR, document chunking, review workflow, and embedding operations that are heavier than a simple warehouse build.
- **Evidence to confirm:** A two-week prototype reaches p95 retrieval latency under 900 ms and lifts grounded-answer pass rate above 85% on the field-service test set.

- **Evidence to kill:** Document access restrictions or scan quality make fewer than 60% of critical procedures extractable into a reliable governed corpus.

Option 3. Delay the assistant and first build a feature pipeline for failure-risk scoring from tickets, sensors, and work-order history.

- **Expected impact:** This may yield faster operational value for dispatch planning but does not satisfy the VP's Q4 requirement for technician self-service answers.
- **Key risk:** The platform solves a different problem than the committed business goal and loses executive support despite technical quality.
- **Evidence to confirm:** Dispatch leaders rank failure prediction above answer retrieval and agree to change the quarter's success criteria in writing.
- **Evidence to kill:** The original success metric remains first-call fix improvement through grounded procedure answers rather than forecast accuracy.

Option 4. Buy a generic SaaS knowledge assistant and ingest everything reachable, leaving governance and source controls for later.

- **Expected impact:** This appears to compress delivery time but shifts risk into licensing, privacy, and unverifiable retrieval behavior.
- **Key risk:** Uncontrolled ingestion exposes restricted manuals and outdated procedures, creating safety and compliance exposure for HarborOps.
- **Evidence to confirm:** The vendor proves document-level access control, source citations, retention guarantees, and test-set performance on HarborOps content before contract signature.
- **Evidence to kill:** The vendor cannot isolate restricted manuals or cannot show reproducible grounding scores on a HarborOps evaluation set.

Analysis Walkthrough

The tempting move is Option 1. HarborOps already had Snowflake, dbt, and clean access to the maintenance ticket export plus ERP asset tables. Reachable, owned, nightly refreshed. It looked mature. It was also the wrong center of gravity for the product they had promised.

Start with the product, not the tools. Priya Natarajan, the staff data engineer, was not building a generic analytics mart. She was on the hook for an internal technician assistant that had to answer, with citations, questions like "Which seal kit applies to PumpLine HX-44 after the Sep 2024 bulletin?" Ticket rows can hint at recurring issues; they cannot substitute for the bulletin itself. That is the whole AI-ready versus reachable trap.

The exhibit table gives away the pattern. The SharePoint document repository held 186,000 PDFs, DOCX files, and scanned vendor manuals. Messy, yes. But it contained the actual procedures, bulletins, and exception notes technicians use in the field. Its operational weaknesses were obvious too: three content owners, irregular updates, 14% failed OCR on scanned PDFs, and mixed licensing on vendor documents. That makes it **usable with controls**, not automatically usable. The controls are the architecture.

Option 2 is the strongest fit because it matches the required data product: a governed corpus feeding a RAG pipeline. The lakehouse-first choice is not ideological here; it follows from the state changes HarborOps must manage. Raw documents land unchanged, controlled processing strips duplicates and flags licensing/privacy issues, curated chunks are versioned with lineage, and only approved content reaches retrieval. Those boundaries are difficult to express cleanly in a warehouse-first design centered on tabular analytics, especially when human review must interrupt automation.

The hardest tradeoff is complexity. A governed corpus is not free. Priya's team must operate OCR, chunking, metadata extraction, embedding refresh, and review queues. But the case numbers make that acceptable. The target was 12,000 monthly technician queries and a Q4 pilot over one business unit, not internet scale. Right-sized scope means one lakehouse, one vector index, one governed corpus, and a narrow evaluation set of 120 technician questions. No multi-region failover theater. No speculative training release. Enough to build next.

Option 3 is defensible only if HarborOps changes the business goal. The module's discipline matters here: feature pipeline, training release, governed corpus, and RAG pipeline are different products. A failure-risk model could be useful later, but it does not answer the committed need for grounded service guidance. Good engineering that solves the wrong product still fails the

review.

Option 4 should be rejected fastest. The source assessment shows too many operational-contract risks: vendor-manual licensing ambiguity, maintenance SOPs that include personal phone numbers in attachments, and terminology drift across acquired fleets. A black-box SaaS assistant might demo well, but without document-level governance, version visibility, and reproducible testing, HarborOps would be claiming readiness on confidence instead of evidence.

The storage ADR should therefore record a compromise: keep Snowflake as the analytics warehouse of record for ERP, ticket, and reporting workloads, but add a lakehouse-centered controlled corpus path for unstructured service content. That is better than pretending one storage pattern should dominate every product. It also mirrors the evidence: structured data remains useful as retrieval filters and answer context, while documents carry the semantic payload.

If I were signing the ADR, I would pick Option 2 and explicitly accept one tradeoff: more implementation complexity in exchange for materially lower answer-risk and stronger governance. The validation plan would be concrete: classify sources, prototype OCR on the top 20,000 documents, measure extraction success, run a 120-question retrieval evaluation, and block go-live unless grounded-answer pass rate exceeds 85% and restricted-document leakage is zero.

